DATABASE MANAGEMENT SYSTEMS

PROJECT REPORT

HOTEL MANAGEMENT SYSTEM

HMS

A database-driven internal web application for managing hotel rooms, customers, bookings, check-in/check-out operations, payments, and operational reports.

Submitted by	M V Sri Navin Shenbagan, Gajjala Sri Charan Reddy, Athreya Srinivas
Student ID(s)	CS25I1012, CS25I1035, CS25I1038
Department / Programme	CSE
Institution	Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram
Course / Subject	Database Management Systems
Submitted to	Dr. Ratna Raju Mekala
Academic year	2026-2027

REPORT CONTENTS

Section	Topics Covered
1. Project Overview	Project title, problem statement, objectives, scope, and system boundary
2. Application Architecture	Flask/PostgreSQL stack, three-tier architecture, request lifecycle, DB configuration
3. Functional Modules	Staff auth, dashboard, rooms, customers, bookings, check-in/out, payments, reports
4. Database Design	Normalized 5-table schema, integrity constraints, ER Diagram, Relational Schema
5. Important SQL Queries	DDL constraints, B-tree indexes, FOR UPDATE overlap checks, FILTER/LATERAL reports
6. App–Database Interaction	psycopg2 context managers, read/write workflows, transactions, access control
7. Application Screenshots	Structured capture templates for Login, Dashboard, Rooms, Customers, Bookings, Payments
8. Results and Observations	Seed data metrics, verified analytical outputs, automated pytest verification
9. Conclusion and References	Summary of work, source file inventory, GitHub repository and live deployment

Documentation note: Every table, SQL query, constraint, and route documented in this report is verified directly against the project source files in *M-V-SRI-NAVIN-SHENBAGAN/hotel-management* (PostgreSQL 16.2 + Flask + psycopg2) and validated via the automated end-to-end test suite.

1. PROJECT OVERVIEW

1.1 Project title

Hotel Management System (HMS)

1.2 Problem statement

Manual front-desk logs and disconnected spreadsheets make day-to-day hotel operations error-prone. Common operational bottlenecks include double-booking the same room over overlapping date intervals, losing track of room availability between check-in and check-out, mismatched night and tariff totals, fragmented guest stay histories, and incomplete visibility into settled versus pending payments.

To solve these challenges, the Hotel Management System (HMS) centralizes room inventory, customer records, reservations, front-desk arrivals/departures, payment tracking, and tabular management reporting inside a single transactional PostgreSQL-backed web application.

1.3 Objectives

- Design a normalized PostgreSQL relational schema enforcing entity, referential, uniqueness, and domain integrity.
- Build a clean internal web portal using Flask, Jinja2 HTML templates, CSS, and lightweight client-side JavaScript.
- Protect administrative workflows behind session-based authentication using PBKDF2-SHA256 salted password hashes.
- Prevent double-booking races by locking room rows and overlapping reservation intervals with `SELECT ... FOR UPDATE` inside atomic transactions.
- Synchronize room `availability_status` cleanly across check-in, check-out, edit, and cancellation workflows.
- Compute real-time occupancy KPIs, outstanding booking balances, monthly revenue, room-type demand, and guest history directly from live SQL queries.

1.4 Project scope

HMS is scoped strictly as an internal hotel operations portal for administrators and front-desk reception staff. Authenticated staff manage rooms, register guests, create/edit/cancel bookings, process check-ins and check-outs, record partial or full payments, and review structured tabular reports.

> **Scope & Architecture Boundary**
> HMS focuses exclusively on internal hotel staff operations—it intentionally omits public commercial marketing pages or simulated/hardcoded demo statistics. All dashboard figures, availability checks, and reports are computed live from PostgreSQL using parameterized `psycopg2` queries.

2. APPLICATION DESCRIPTION AND ARCHITECTURE

2.1 Description of the application

The application follows a modular server-rendered MVC-style Python architecture. HTTP routing and Jinja2 template rendering reside in `app.py`, business rules and SQL orchestration live in `services.py`, database connection/transaction context managers live in `db.py`, and environment configuration is centralized in `config.py`. All user-facing views inherit a unified sidebar layout in `templates/base.html` styled by `static/css/style.css` and enhanced by `static/js/app.js`.

2.2 Technology stack in the repository

Layer	Technology / Library	Role in HMS Repository
Presentation	Jinja2 HTML5, CSS3, Vanilla JS	Sidebar layout, forms, tabular reports, live night/total calculation, /api/rooms/available fetch
Web Framework	Flask 3.x + Werkzeug	Application factory (create_app), @login_required guard, ProxyFix, PBKDF2 password hashing
Business Logic	Python services.py	Validation, atomic booking/check-in transactions, overlap locking, payment balance enforcement
Database Driver	psycopg2-binary (RealDictCursor)	Direct parameterized SQL execution (%s placeholders) with zero ORM overhead
Relational DBMS	PostgreSQL 16	ACID transactions, CHECK/UNIQUE/FK constraints, B-tree indexes, LATERAL & DATE_TRUNC queries
Automated Testing	pytest + Flask test_client	8 end-to-end integration tests verifying auth, CRUD, concurrency guards, rollbacks, and APIs

2.3 Three-tier architecture

1. Presentation Tier	2. Application Tier	3. Data Tier
Staff Browser + Jinja2 Templates • Sidebar navigation & forms • `style.css` responsive tables • `app.js` dynamic room fetch	Flask + Business Services • `app.py` routes & auth guard • `services.py` domain rules • `db.py` tx context managers	PostgreSQL 16 Database • 5 normalized relational tables • PK / FK / CHECK / UNIQUE rules • Row locks (`FOR UPDATE`)

End-to-End Request Lifecycle

Step 1 (HTTP & Session Guard): Browser sends GET/POST request; `ProxyFix` and `@login_required` verify authenticated staff session.
Step 2 (Transactional Service): Route calls `services.py`, opening a `db_cursor()` or atomic `transaction()` block that executes parameterized `psycopg2` queries.
Step 3 (Commit & Render): PostgreSQL commits or rolls back on constraint/validation error; Flask flashes feedback or renders Jinja2 HTML.

2.4 Database configuration and initialization

Connection parameters are read from `DATABASE_URL` (used on managed cloud hosts such as Render) or discrete variables (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`) defined in `.env` and loaded by `config.py`. Running `python3 create_db.py --reset` executes `sql/schema.sql` and `sql/seed.sql` idempotently; on startup, `_ensure_initialized()` automatically provisions missing tables and seed rows when `AUTO_INIT_DB=1`.

3. FUNCTIONAL MODULES

3.1 Staff authentication and session security (/login, /logout)

Verifies staff credentials against admins.password_hash using Werkzeug check_password_hash (PBKDF2-SHA256). All operational routes are protected by @login_required. AdaptiveSessionInterface and ProxyFix automatically set secure cookie attributes across both HTTP local development and HTTPS reverse-proxy deployments.

3.2 Dashboard overview (/dashboard)

Displays live database KPIs—Total Rooms, Available Rooms, Occupied Rooms, Maintenance Rooms, Active Bookings, Today's Check-ins, Today's Check-outs, and Total Collected Revenue—alongside scheduled arrival and departure tables.

3.3 Room management (/rooms, /rooms/new, /rooms/<id>/edit)

Supports creating, editing, searching by room number, filtering by room_type (Single, Double, Deluxe, Suite) and availability_status (Available, Occupied, Maintenance), and safe deletion guarded by ON DELETE RESTRICT plus an active check-in check.

3.4 Customer management (/customers, /customers/new, /customers/<id>)

Registers and updates guests with UNIQUE constraints on phone_number and email, keyword search across name/phone/email, and a dedicated Customer Detail view aggregating stay history, billed amounts, paid amounts, and remaining balances.

3.5 Booking management (/bookings, /bookings/new, /bookings/<id>/edit, /api/rooms/available)

Allows selecting an existing customer or registering a new customer inline within the same database transaction. Enforces check_out_date > check_in_date, auto-calculates num_nights and total_amount from authoritative room rates, queries /api/rooms/available dynamically, and prevents overlapping bookings via FOR UPDATE row locks.

3.6 Check-in / Check-out desk (/checkin-checkout)

Lists Confirmed reservations awaiting arrival and Checked-in stays ready for departure. Check-in marks the booking 'Checked-in' and the room 'Occupied'; check-out marks the booking 'Checked-out' and restores the room to 'Available' while leaving future Confirmed bookings intact.

3.7 Payment management (/payments, /payments/new, /payments/<id>/mark-paid)

Records payments across Cash, Card, and UPI with status Paid, Partially Paid, or Pending. Enforces balance checks so cumulative Paid + Partially Paid entries never exceed booking total_amount, and provides one-click Pending settlement.

3.8 Operational reports (/reports)

Provides tabular summaries of total & pending revenue, booking status distribution, occupancy rate, monthly revenue via DATE_TRUNC('month', payment_date), room-type demand/revenue, customer billing history, and live room occupancy via LATERAL subqueries.

4. DATABASE DESIGN

4.1 Tables in the database

Table Name	Primary Key	Purpose & Key Attributes
admins	admin_id (SERIAL)	Staff accounts: unique username, PBKDF2 password_hash, full_name, created_at.
rooms	room_id (SERIAL)	Inventory: unique room_number, room_type, price_per_night > 0, availability_status, description.
customers	customer_id (SERIAL)	Guest profiles: full_name, unique phone_number, unique email, address, created_at.
bookings	booking_id (SERIAL)	Reservations linking customer_id & room_id with dates, num_nights, total_amount, booking_status.
payments	payment_id (SERIAL)	Payment ledger referencing booking_id with amount > 0, payment_date, payment_method, payment_status.

4.2 Main relationships

- **customers → bookings (1 : N):** One customer can place multiple reservations over time; `bookings.customer_id` references `customers.customer_id` with `ON DELETE RESTRICT`.

- **rooms → bookings (1 : N):** One room can hold many non-overlapping reservations; `bookings.room_id` references `rooms.room_id` with `ON DELETE RESTRICT`.
- **bookings → payments (1 : N):** One booking can receive advance, partial, or full settlements; `payments.booking_id` references `bookings.booking_id` with `ON DELETE RESTRICT`.
- **admins (standalone authentication table):** Decoupled from operational guest tables so staff access audits remain independent.

4.3 Integrity constraints and transactional consistency

- **Domain CHECK constraints:** `room_type` in (Single, Double, Deluxe, Suite); `availability_status` in (Available, Occupied, Maintenance); `booking_status` in (Confirmed, Checked-in, Checked-out, Cancelled); `payment_method` in (Cash, Card, UPI); `payment_status` in (Paid, Partially Paid, Pending).
- **Temporal & numeric constraints:** `chk_check_out_after_check_in` guarantees `check_out_date > check_in_date`; `num_nights > 0`; `price_per_night > 0`; `payments.amount > 0`.
- **Concurrency & normalization (3NF):** Every non-key column depends strictly on its primary key (3NF). Server-side transactions lock target rows with `FOR UPDATE` to prevent concurrent overlapping bookings or overpayments.

4.3.1 Database index and key-constraint summary

Index / Constraint	Table(s)	Columns & Purpose
idx_rooms_type_status	rooms	(room_type, availability_status) — speeds up room inventory filters and availability scans.
idx_bookings_room_dates	bookings	(room_id, check_in_date, check_out_date) — accelerates overlap detection and /api/rooms/available.
idx_bookings_customer	bookings	(customer_id) — optimizes per-customer stay history and billing aggregation.
idx_bookings_status	bookings	(booking_status) — accelerates dashboard KPIs and Check-in/Check-out queues.
idx_payments_booking	payments	(booking_id) — fast SUM() rollup per booking for outstanding balance checks.
idx_payments_date_status	payments	(payment_date, payment_status) — supports monthly revenue reports via DATE_TRUNC.
ON DELETE RESTRICT	bookings, payments	Prevents deleting rooms/customers with bookings or bookings with payment records.

4.4 Entity-Relationship (ER) diagram

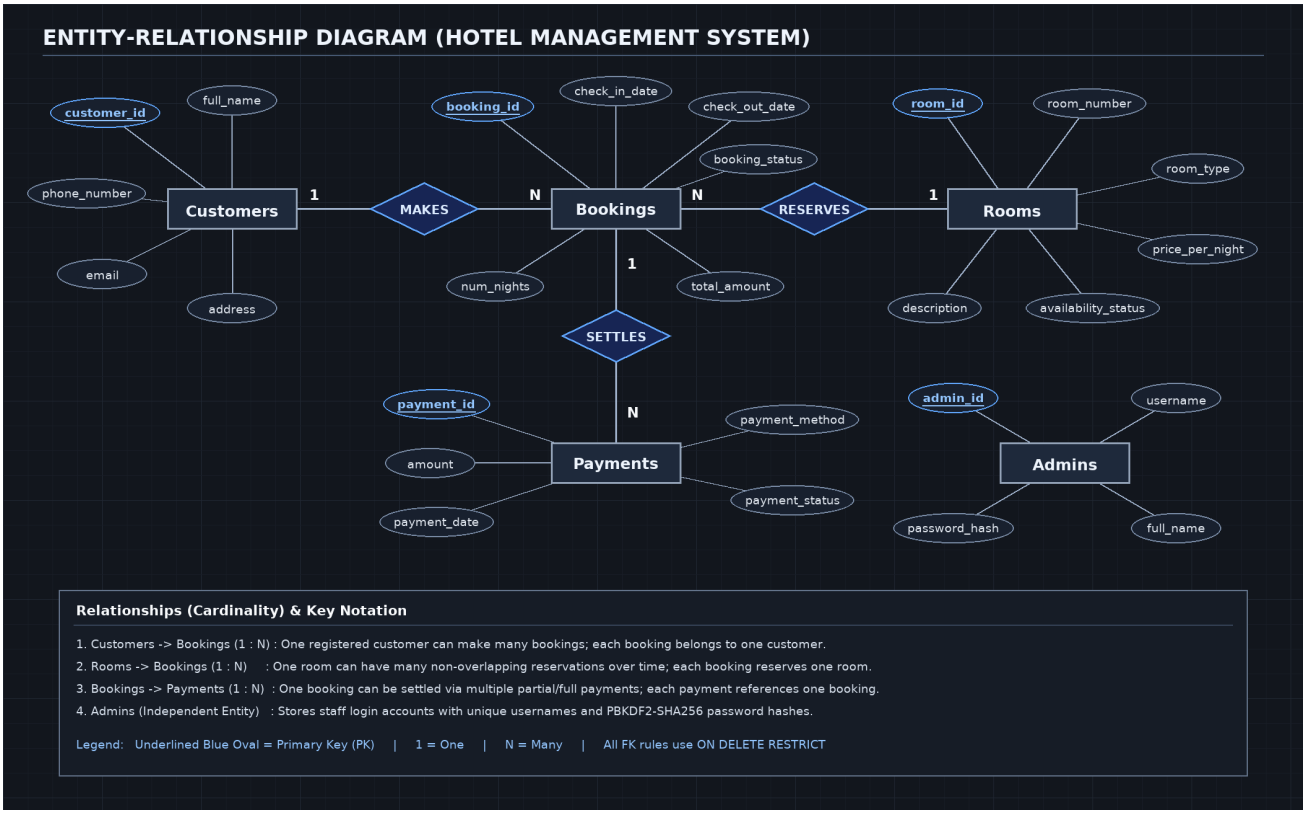


Figure 4.1: Chen ER diagram for the Hotel Management System showing all 5 entities, attributes, PKs, and cardinalities.

4.5 Relational schema

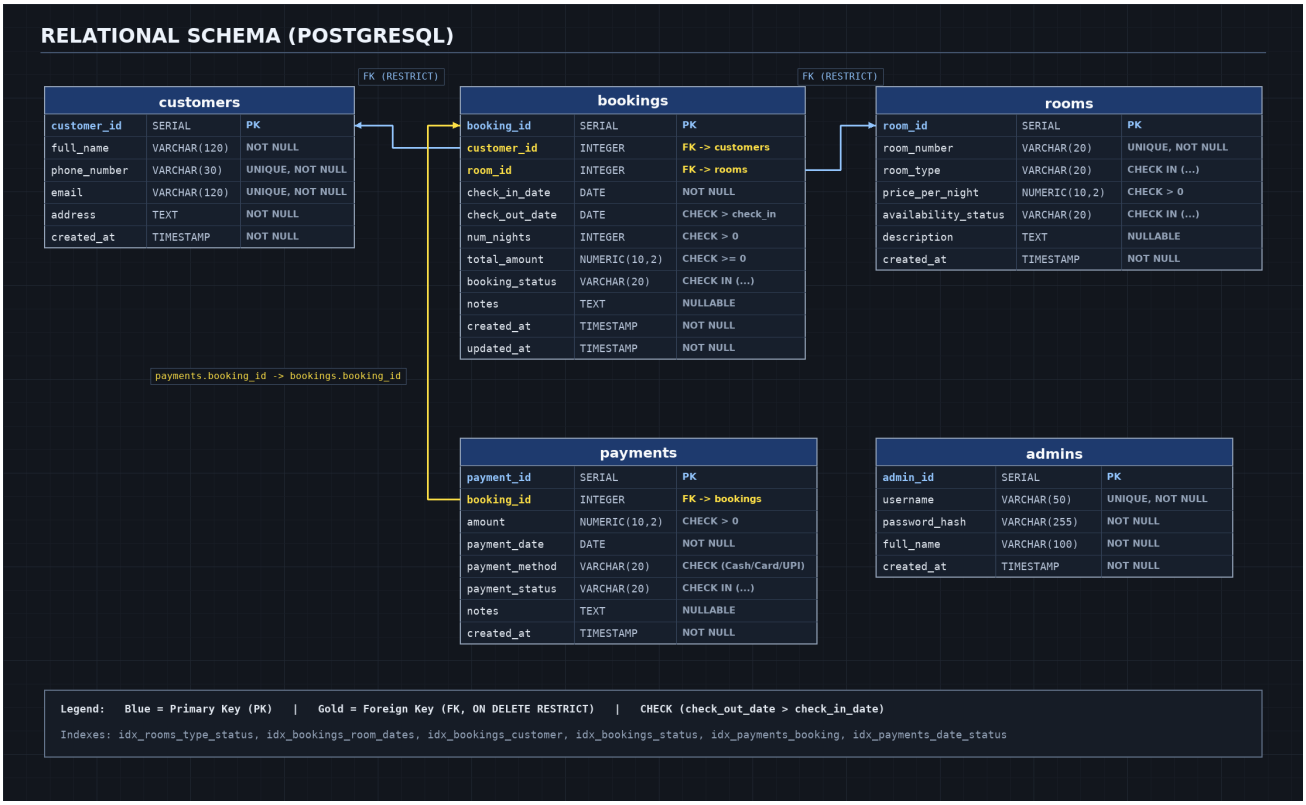


Figure 4.2: PostgreSQL relational schema showing PKs, FKs (ON DELETE RESTRICT), data types, CHECK constraints, and indexes.

5. IMPORTANT SQL COMMANDS AND QUERIES

All schema objects and runtime queries execute directly on PostgreSQL via parameterized SQL statements in `sql/schema.sql`, `sql/seed.sql`, and `services.py`.

5.1 Core table DDL and integrity constraints (sql/schema.sql)

```
CREATE TABLE IF NOT EXISTS rooms (
    room_id             SERIAL PRIMARY KEY,
    room_number         VARCHAR(20) NOT NULL UNIQUE,
    room_type           VARCHAR(20) NOT NULL
                        CHECK (room_type IN ('Single', 'Double', 'Deluxe', 'Suite')),
    price_per_night     NUMERIC(10, 2) NOT NULL CHECK (price_per_night > 0),
    availability_status VARCHAR(20) NOT NULL DEFAULT 'Available'
                        CHECK (availability_status IN ('Available', 'Occupied', 'Maintenance')),
    description         TEXT,
    created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS bookings (
    booking_id     SERIAL PRIMARY KEY,
    customer_id    INTEGER NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT,
    room_id        INTEGER NOT NULL REFERENCES rooms(room_id) ON DELETE RESTRICT,
    check_in_date  DATE NOT NULL,
    check_out_date DATE NOT NULL,
    num_nights     INTEGER NOT NULL CHECK (num_nights > 0),
    total_amount   NUMERIC(10, 2) NOT NULL CHECK (total_amount >= 0),
    booking_status VARCHAR(20) NOT NULL DEFAULT 'Confirmed'
                   CHECK (booking_status IN ('Confirmed','Checked-in','Checked-out','Cancelled')),
    notes          TEXT,
    created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_check_out_after_check_in CHECK (check_out_date > check_in_date)
);

CREATE TABLE IF NOT EXISTS payments (
    payment_id     SERIAL PRIMARY KEY,
    booking_id     INTEGER NOT NULL REFERENCES bookings(booking_id) ON DELETE RESTRICT,
    amount         NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    payment_date   DATE NOT NULL DEFAULT CURRENT_DATE,
    payment_method VARCHAR(20) NOT NULL CHECK (payment_method IN ('Cash', 'Card', 'UPI')),
    payment_status VARCHAR(20) NOT NULL
                   CHECK (payment_status IN ('Paid', 'Partially Paid', 'Pending')),
    notes          TEXT,
    created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

5.2 Performance indexes and sequence synchronization

```
CREATE INDEX IF NOT EXISTS idx_rooms_type_status    ON rooms (room_type, availability_status);
CREATE INDEX IF NOT EXISTS idx_bookings_room_dates  ON bookings (room_id, check_in_date, check_out_date);
CREATE INDEX IF NOT EXISTS idx_bookings_customer    ON bookings (customer_id);
CREATE INDEX IF NOT EXISTS idx_bookings_status      ON bookings (booking_status);
CREATE INDEX IF NOT EXISTS idx_payments_booking     ON payments (booking_id);
CREATE INDEX IF NOT EXISTS idx_payments_date_status ON payments (payment_date, payment_status);

-- Sequence synchronization after deterministic seed IDs (sql/seed.sql)
SELECT setval(pg_get_serial_sequence('bookings', 'booking_id'),
              COALESCE((SELECT MAX(booking_id) FROM bookings), 1));
```

5.3 Transactional room locking and overlap prevention (services.py)

```
-- Lock candidate room row to serialize concurrent reservation requests
SELECT room_id, room_number, price_per_night, availability_status
FROM rooms
WHERE room_id = %s
FOR UPDATE;

-- Detect overlapping Confirmed or Checked-in reservations on [check_in, check_out)
SELECT booking_id, check_in_date, check_out_date, booking_status
FROM bookings
WHERE room_id = %s
  AND booking_status IN ('Confirmed', 'Checked-in')
  AND check_in_date < %s
  AND check_out_date > %s
ORDER BY check_in_date ASC
FOR UPDATE;

-- Insert booking using server-computed nights and room tariff
INSERT INTO bookings (
    customer_id, room_id, check_in_date, check_out_date,
    num_nights, total_amount, booking_status, notes
)
VALUES (%s, %s, %s, %s, %s, %s, 'Confirmed', %s)
RETURNING booking_id;
```

5.4 Live dashboard KPI aggregation query

```
-- Dynamic dashboard KPI query in a single round-trip (services.get_dashboard_data)
SELECT
    (SELECT COUNT(*) FROM rooms) AS total_rooms,
    (SELECT COUNT(*) FROM rooms WHERE availability_status = 'Available') AS available_rooms,
    (SELECT COUNT(*) FROM rooms WHERE availability_status = 'Occupied') AS occupied_rooms,
    (SELECT COUNT(*) FROM rooms WHERE availability_status = 'Maintenance') AS maintenance_rooms,
    (SELECT COUNT(*) FROM bookings
     WHERE booking_status IN ('Confirmed', 'Checked-in')) AS active_bookings,
    (SELECT COUNT(*) FROM bookings
     WHERE check_in_date = CURRENT_DATE
       AND booking_status IN ('Confirmed', 'Checked-in')) AS today_checkins,
    (SELECT COUNT(*) FROM bookings
     WHERE check_out_date = CURRENT_DATE
       AND booking_status IN ('Checked-in', 'Checked-out')) AS today_checkouts,
    (SELECT COALESCE(SUM(amount), 0) FROM payments
     WHERE payment_status IN ('Paid', 'Partially Paid')) AS total_revenue;
```

5.5 Monthly revenue and room-type demand aggregation queries

```
-- Monthly collected revenue breakdown (services.get_reports_data)
SELECT
    TO_CHAR(DATE_TRUNC('month', payment_date), 'YYYY-MM') AS month_key,
    TO_CHAR(DATE_TRUNC('month', payment_date), 'Mon YYYY') AS month_label,
    COUNT(*) AS payment_count,
    COALESCE(SUM(amount), 0) AS revenue
FROM payments
WHERE payment_status IN ('Paid', 'Partially Paid')
GROUP BY 1, 2
ORDER BY 1 DESC;

-- Room-type demand and booked value using PostgreSQL FILTER clauses
SELECT
    r.room_type,
    COUNT(DISTINCT r.room_id) AS room_count,
    COUNT(b.booking_id) FILTER (WHERE b.booking_status != 'Cancelled') AS booking_count,
    COALESCE(SUM(b.num_nights) FILTER (WHERE b.booking_status != 'Cancelled'), 0) AS total_nights,
    COALESCE(SUM(b.total_amount) FILTER (WHERE b.booking_status != 'Cancelled'), 0) AS booked_value
FROM rooms r
LEFT JOIN bookings b ON b.room_id = r.room_id
GROUP BY r.room_type
ORDER BY booking_count DESC, booked_value DESC, r.room_type ASC;
```

5.6 Customer balance calculation and LATERAL occupancy query

```
-- Customer booking history with outstanding balance calculation (services.get_customer_bookings)
SELECT
    b.booking_id, b.check_in_date, b.check_out_date, b.num_nights,
    b.total_amount, b.booking_status, r.room_number, r.room_type,
    COALESCE(p.paid_amount, 0) AS paid_amount,
    CASE WHEN b.booking_status = 'Cancelled' THEN 0
         ELSE GREATEST(b.total_amount - COALESCE(p.paid_amount, 0), 0)
    END AS outstanding_balance
FROM bookings b
JOIN rooms r ON r.room_id = b.room_id
LEFT JOIN (
    SELECT booking_id,
           SUM(CASE WHEN payment_status IN ('Paid', 'Partially Paid')
                    THEN amount ELSE 0 END) AS paid_amount
    FROM payments
    GROUP BY booking_id
) p ON p.booking_id = b.booking_id
WHERE b.customer_id = %s
ORDER BY b.check_in_date DESC, b.booking_id DESC;

-- Live room occupancy board using LATERAL subqueries (services.get_reports_data)
SELECT
    r.room_number, r.room_type, r.price_per_night, r.availability_status,
    cur_guest.customer_name AS current_guest,
    cur_guest.check_out_date AS current_check_out,
    next_guest.check_in_date AS next_check_in
FROM rooms r
LEFT JOIN LATERAL (
    SELECT c.full_name AS customer_name, b.check_out_date
    FROM bookings b JOIN customers c ON c.customer_id = b.customer_id
    WHERE b.room_id = r.room_id AND b.booking_status = 'Checked-in'
    ORDER BY b.check_in_date DESC LIMIT 1
) cur_guest ON TRUE
LEFT JOIN LATERAL (
    SELECT b.check_in_date FROM bookings b
    WHERE b.room_id = r.room_id AND b.booking_status = 'Confirmed'
      AND b.check_out_date >= CURRENT_DATE
    ORDER BY b.check_in_date ASC LIMIT 1
) next_guest ON TRUE
ORDER BY r.room_number ASC;
```

6. APPLICATION–DATABASE INTERACTION

6.1 Database connection and context managers (db.py)

`db.py` exposes two context managers built around `psycopg2.extras.RealDictCursor`: `db_cursor(commit=False)` for read queries and `transaction()` (`commit=True`) for state-changing operations. On any Python exception or PostgreSQL constraint violation, the `except` handler invokes `conn.rollback()` before closing the cursor and connection cleanly in `finally`.

6.2 Read and write execution workflows

- **Read workflow:** Route handlers call functions in `services.py` (e.g., `list_rooms`, `get_dashboard_data`, `get_reports_data`), which open `db_cursor()`, pass bound filter parameters, and return dictionary rows directly to Jinja2.
- **Write workflow:** POST handlers submit form inputs to `transaction()`, translate `psycopg2.IntegrityError` (UniqueViolation, ForeignKeyViolation, CheckViolation) into clear `ValidationError` messages, and flash user-friendly alerts.

6.3 Example: atomic booking creation with inline customer

When front-desk staff choose *New Customer* inside `/bookings/new`, `services.create_booking()` opens a single `with transaction() as cur:` block: (1) inserts the new customer with `RETURNING customer_id`, (2) locks the requested room row via `FOR UPDATE` and rejects rooms in Maintenance, (3) queries overlapping Confirmed/Checked-in stays with `FOR UPDATE`, and (4) computes `num_nights` and `total_amount` server-side before inserting into `bookings`. If step 2 or 3 raises `ValidationError`, the entire transaction—including the inline customer row—rolls back atomically.

6.4 Access control and SQL injection protection

- **Route protection:** Every non-login route carries `@login_required`, redirecting unauthenticated requests to `/login`.
- **Password security:** `sql/seed.sql` stores PBKDF2-SHA256 hashes (1,000,000 iterations) verified via `check_password_hash`.
- **SQL injection safety:** Every dynamic filter, search string, ID, and update payload is passed exclusively through `psycopg2` `%s` query placeholders—never string concatenation.

6.5 Database exception translation & transaction guarantees

PostgreSQL / Domain Condition	Database / Service Trigger	Handled Application Behavior
UniqueViolation (23505)	Duplicate room_number, phone_number, or email	_handle_integrity_error() rolls back tx and flashes field-specific ValidationError.
ForeignKeyViolation (23503)	Deleting referenced room or customer	ON DELETE RESTRICT blocks orphan deletion; staff sees clear dependency warning.
CheckViolation (23514)	Invalid status, amount <= 0, check_out <= check_in	Database CHECK constraint rejects invalid state; rolled back immediately.
Overlap / Overpayment Guard	Date collision or payment > remaining balance	FOR UPDATE lock serializes concurrent writes; ValidationError aborts transaction.

6.6 Primary HTTP routes mapped to service functions

Route Pattern	Service Function(s)	Transaction Mode
/login, /logout	authenticate_admin()	Read-only query + PBKDF2 verify
/dashboard	get_dashboard_data()	Single-session read-only KPIs
/rooms, /rooms/<id>/edit, /rooms/<id>/delete	list_rooms, create_room, update_room, delete_room	Atomic tx + FOR UPDATE check
/customers, /customers/<id>	list_customers, create_customer, get_customer_bookings	Read / atomic tx insert & update
/bookings/new, /api/rooms/available	create_booking, get_available_rooms	Atomic tx + room/overlap locks
/checkin-checkout, /bookings/<id>/check-in|out	check_in_booking, check_out_booking	Atomic tx syncing room & booking
/payments, /payments/new, /payments/<id>/mark-paid	list_payments, create_payment, mark_payment_paid	Atomic tx + balance check
/reports	get_reports_data()	Multi-query tabular SQL reports

7. APPLICATION SCREENSHOTS / OUTPUTS

Capture clear screenshots from the running HMS application (local or deployed at `https://hotel-management-xb2u.onrender.com`) into the designated frames below. Keep browser text legible and avoid exposing sensitive credentials in screenshots.

Screenshot 1: Staff Login Screen

INSERT SCREENSHOT 1

`Route: /login`

Verify clean internal HMS login card, username/password fields, and validation feedback.

Screenshot 2: Dashboard Overview

INSERT SCREENSHOT 2

`Route: /dashboard`

Verify 8 live operational KPI cards plus Today's Check-ins and Today's Check-outs tables.

Screenshot 3: Room Management & Filters

INSERT SCREENSHOT 3

`Route: /rooms`

Verify room table, room_type / availability_status filters, Add Room, Edit, and Delete actions.

Screenshot 4: Customer Registry & Booking History

INSERT SCREENSHOT 4

`Route: /customers & /customers/<id>`

Verify guest search, unique email/phone validation, and per-customer outstanding balances.

Screenshot 5: New Booking & Check-in / Check-out Desk

INSERT SCREENSHOT 5

Route: /bookings/new & /checkin-checkout

Verify inline customer toggle, dynamic room availability lookup, auto total, and Check-in/Out buttons.

Screenshot 6: Payment Ledger & Operational Reports

INSERT SCREENSHOT 6

Route: /payments & /reports

Verify Cash/Card/UPI payment history, Mark Paid action, monthly revenue, and room-type summary tables.

8. RESULTS AND OBSERVATIONS

8.1 Seed data present in sql/seed.sql

Table	Record Count	Seeded Breakdown & Relative Dates
admins	2	Default accounts: admin ('Hotel Administrator') and reception ('Front Desk Staff') with PBKDF2 hashes.
rooms	10	Rooms 101–402 across Single (3), Double (3), Deluxe (2), Suite (2); 5 Available, 4 Occupied, 1 Maintenance.
customers	8	Registered guests with unique phone numbers, emails, and residential/business addresses.
bookings	13	5 Checked-out, 4 Checked-in (2 due out today), 3 Confirmed (2 arriving today), 1 Cancelled.
payments	10	6 Paid, 3 Partially Paid, 1 Pending across Card, UPI, and Cash spanning multiple months.

8.2 Verified outputs derived from seed records

Table 8.1 — Room Type Demand & Booked Value (excluding Cancelled booking #13; matching `services.get_reports_data()`):

Room Type	Rooms	Valid Bookings	Total Nights	Booked Value
Deluxe	2	4	11	$2,310.00
Double	3	3	9	$1,215.00
Single	3	3	8	$680.00
Suite	2	2	7	$2,460.00
Total (All Valid)	**10**	**12**	**35**	**$6,665.00**

Table 8.2 — Customer Billing & Outstanding Balances (matching `customer_history` query in `services.get_reports_data()`):

Customer Name	Total / Valid Bookings	Total Billed	Total Paid	Outstanding
Elena Rostova	2 / 2	$1,650.00	$1,020.00	$630.00
Vikram Sharma	2 / 2	$1,035.00	$630.00	$405.00
Aarav Mehta	2 / 2	$885.00	$885.00	$0.00
Priya Nair	2 / 2	$575.00	$490.00	$85.00
Hannah Lim	1 / 1	$1,440.00	$0.00	$1,440.00
Meera Krishnan	1 / 1	$420.00	$420.00	$0.00
Karthik Subramanian	1 / 1	$405.00	$200.00	$205.00
Daniel Brooks	2 / 1 (1 Cancelled)	$255.00	$100.00	$155.00

8.3 Verification and observations

- **Database initialization verified:** Running `python3 create_db.py --reset` against PostgreSQL 16.2 created all 5 tables, 6 indexes, and seeded 2 admins, 10 rooms, 8 customers, 13 bookings, and 10 payments with zero errors.

- **End-to-end automated test suite (8/8 passed):** `pytest -v` verified login/logout guards, room CRUD & deletion restriction, customer creation & uniqueness checks, overlapping booking rejection, inline customer transaction rollback on unavailable rooms, check-in/out room status sync, payment overpayment prevention & settlement, reports totals ($3,745.00 collected / $1,440.00 pending), and proxy session cookie handling.
- **Cloud deployment readiness:** The application runs in production via `app.py` + PostgreSQL on Render (`hotel-management-xb2u.onrender.com`) with automatic schema verification on boot.

9. CONCLUSION AND REFERENCES

9.1 Conclusion

The Hotel Management System (HMS) successfully demonstrates how normalized PostgreSQL schema design, strict relational constraints, row-level locking, and clean Flask backend architecture work together to power reliable hotel front-desk operations. By enforcing temporal booking validation, atomic check-in/check-out synchronization, and balance-checked payment records directly at the database and service boundaries, HMS eliminates common manual booking conflicts while giving staff real-time operational visibility.

9.2 References and source files

File / Resource	Purpose
sql/schema.sql & sql/seed.sql	PostgreSQL table definitions, CHECK/UNIQUE/FK constraints, B-tree indexes, and idempotent seed script.
db.py & create_db.py	psycopg2 connection/transaction context managers, automatic schema bootstrap, and CLI reset utility.
services.py	Transactional business logic, FOR UPDATE overlap locking, payment settlement, and reporting SQL queries.
app.py & config.py	Flask create_app factory, @login_required guard, ProxyFix, routes, /api/rooms/available, environment config.
templates/ & static/	14 Jinja2 HTML templates, responsive CSS stylesheet (style.css), and vanilla JS interactions (app.js).
tests/test_app.py	8 automated pytest integration tests covering all modules, transactional rollbacks, and SQL rules.
Repository & Live Deployment	github.com/M-V-SRI-NAVIN-SHENBAGAN/hotel-management | hotel-management-xb2u.onrender.com